Peyton Bailey

January 11th, 2026

D604- Task One

A. Select one of the scenarios and describe the purpose of this data analysis by doing the following:

1. Provide one research question that you will answer using neural network models and computer vision techniques. Be sure the research question is relevant to a real-world organizational situation related to the images, video, and audio captured in your chosen dataset.

Can a CNN accurately classify twelve plant seedling species from 64x64 RGB images to enable automated weed detection in precision agriculture?"

2. Define the objectives or goals of the data analysis. Be sure *each* objective or goal is reasonable within the scope of the research question and is represented in the available data.

The goals of this data analysis are to achieve at least 85% test accuracy across twelve classes, minimize false positives on crop species, and demonstrate that the model performs well on unseen field images.

3. Identify an industry-relevant type of neural network capable of performing an image, audio, or video classification task that can be trained to produce useful predictions on image sequences on the selected dataset.

For the image dataset, I will be using a Convolutional Neural Network (CNN).

4. Justify your choice of neural network in part A3.

I selected CNN because they excel at spatial hierarchies in images via convolutional/pooling layers. CNNs also utilize locally connected layers vs fully connected ones. The neurons focus on examining only small patches of the image, rather than the entire image. As a result, the network focuses on meaningful neighborhoods rather than treating every pixel as equally related to every other, ultimately making learning easier and more data-efficient.

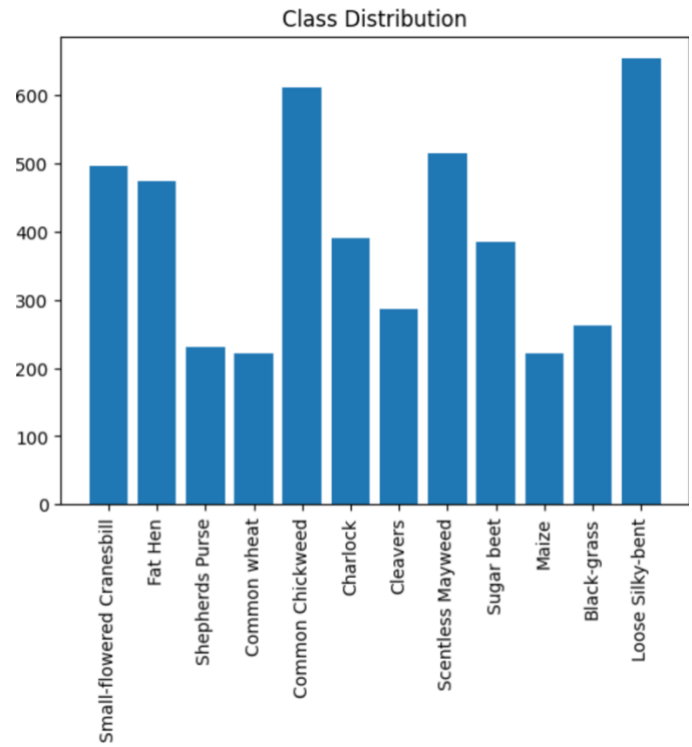
Part II: Data Preparation

If you selected the Image Dataset:

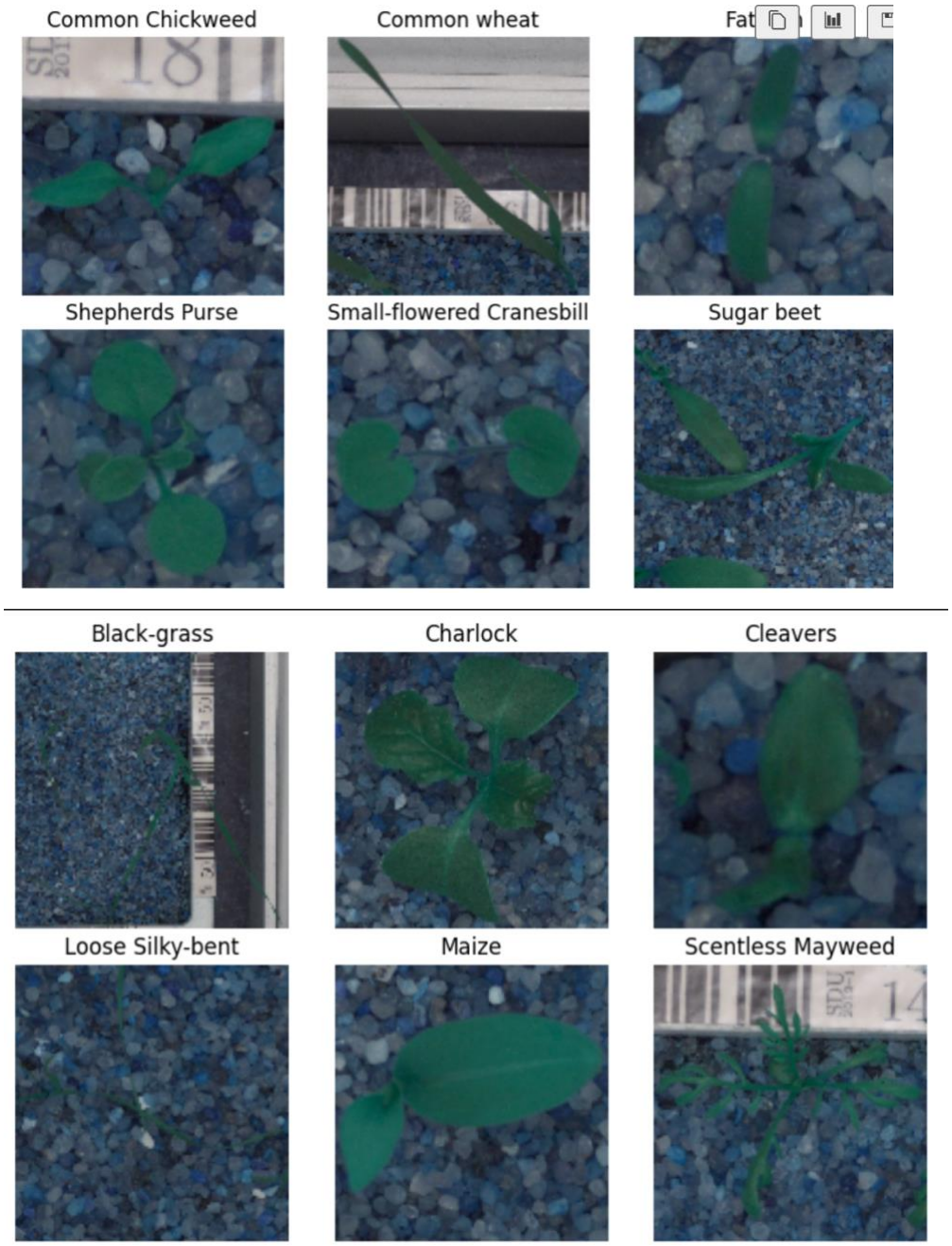
B. Describe the data preparation process by doing the following:

1. Perform exploratory data analysis on the chosen image dataset and provide screenshots of the following:

a. visualization for the distribution of the different classes



b. sample images with associated labels



2. Perform data augmentation and justify the steps taken to augment the images.

Data augmentation is the process of generating more data by performing random transformations to the original images. The transformations I used on my training data were random horizontal flips and random 45-degree rotations. The purpose of this is to increase the size and diversity of

the training set, which ultimately improves model robustness and reduces overfitting (DataCamp, 2024). These transforms were assigned to the variable "train_transform".

3. Normalize the images and discuss the steps taken for normalization.

Normalization is the process of scaling input data by subtracting the mean and dividing by the standard deviation for each channel, in this case, the three color channels (R, G, B). This was also included with the previous transformation step mentioned before. Normalization helps the optimizer converge more reliably and prevents channels with larger raw values from dominating the gradients. The values for the mean and standard deviation are derived from the statistics of a large dataset called ImageNet. This serves as a natural default for natural RGB photos, and using them makes the seedling images appear statistically similar to the data for which many CNN architectures are designed (Geeks, 2022). This improves training stability and typically leads to better generalization performance on the validation and test sets.

4. Perform a train-validation-test split and justify your selection of the proportions for the split.

I split the data into 2850 training images, 950 validation images, and 950 test images (60% / 20% / 20%). This provides the model with sufficient data to learn its parameters while maintaining two substantial held-out sets. The validation set is sufficiently large to reliably select hyperparameters and stopping criteria. The equally sized test set provides an unbiased estimate of final performance after all modeling decisions are finalized.

5. Encode the target feature appropriately for all your datasets and discuss the steps taken.

The first step to encoding the target feature was to import the "LabelEncoder" from "sklearn.preprocessing". Then, I instantiated the "LabelEncoder" class by creating an object: "le = LabelEncoder()". Next, I fitted the encoder on the original string class labels and

transformed them into integer class IDs: “y_encoded = le.fit_transform(labels[“Label”)”. I used these integer labels for all three splits (train, validation, test) so the network could learn to predict class indices 0–11. After training, I used “le.inverse_transform()” to convert predicted indices back to the original plant names when computing per-class metrics and plotting the confusion matrix.

6. Provide a copy of all your datasets.

Part III: Network Architecture

E. Describe the type of network used by doing the following:

1. Provide the output of the model summary.

```
Net(
  (feature_extractor): Sequential(
    (0): Conv2d(3, 16, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (1): ReLU()
    (2): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (3): Conv2d(16, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (4): ReLU()
    (5): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (6): Flatten(start_dim=1, end_dim=-1)
  )
  (classifier): Linear(in_features=8192, out_features=12, bias=True)
)
Total trainable parameters: 103404
```

2. Discuss components of your neural network architecture and justify the choice of the following:

a. number of layers

The network comprises a small feature extractor with two convolutional layers and one linear output layer, resulting in a total of three learnable layers. This is deep enough to learn useful visual features from 64×64 images, but not so deep that it would be hard to train on the relatively small seedling dataset.

b. types of layers

The feature extractor utilizes convolution, ReLU activation functions, and max-pool blocks to capture local patterns, including edges, textures, and shapes, allowing for progressive downsampling of the image, a standard approach for image classification. The final classifier is a fully connected linear layer that maps the extracted features to the twelve plant classes.

c. number of nodes per layer

The first convolutional layer uses 16 channels, while the second uses 32. This gradually increases model capacity while keeping computation reasonable. After two pooling steps, the spatial size is sufficiently small that the flattened feature vector has 8192 elements, which are then fed into a single linear layer with 12 outputs (one logit per class).

d. total number of parameters (weights, bias)

There are 103,404 trainable parameters. This is large enough to fit non-trivial patterns in the images, but much smaller than modern, very deep CNNs, which helps reduce overfitting and keeps training time manageable.

e. activation functions (hidden layers, output layer)

ReLU is used after each convolution because it is simple, computationally efficient, and helps gradients flow more effectively in deeper networks. For the output, a linear layer is followed by the softmax function, implicit in the cross-entropy loss, which gives a probability distribution over the 12 classes.

3. Discuss the backpropagation process and justify the choice of the following hyperparameters:

a. loss function

The loss function used was cross-entropy loss for multi-class classification, due to its ability to compare the predicted class probabilities with the one-hot true label, making it a standard for image classifiers.

b. optimizer

Stochastic gradient descent (SGD) updates all network weights in the opposite direction of the gradient with a single global learning rate. On the other hand, the Adam optimizer adapts the learning rate per parameter based on the gradient history (DataCamp, 2025). I compared SGD and Adam in a small grid search over optimizers and learning rates. Since Adam extends SGD by keeping moving averages of past gradients and squared gradients, it can adapt the effective learning rate for each parameter and converge faster on small vision models. In my experiments, Adam with a learning rate of 3×10^{-4} produced a smoother decrease in training and validation loss, and reached higher validation accuracy than any of the SGD settings; therefore, it was selected as the final optimizer.

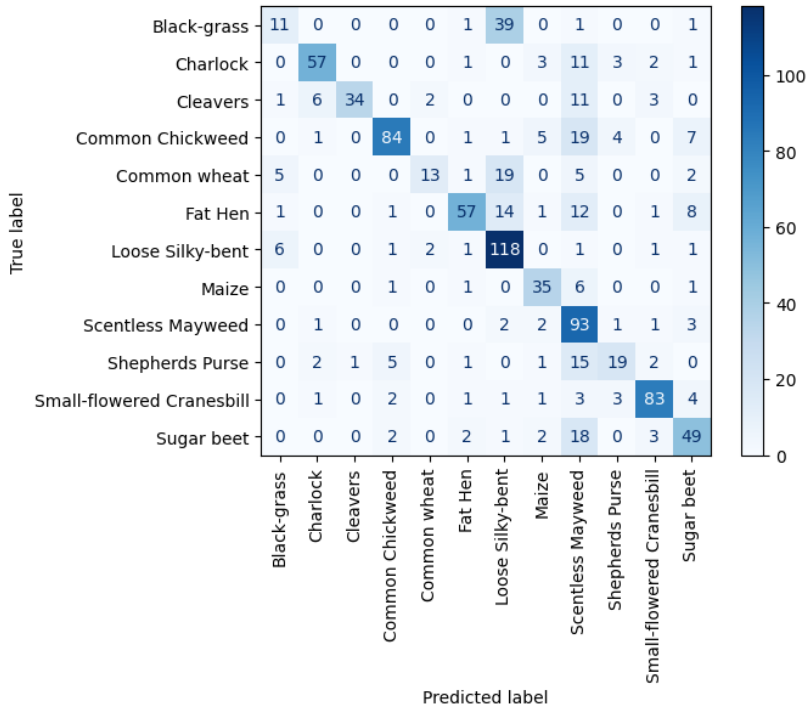
c. learning rate

A learning rate of 3×10^{-4} was selected after conducting a grid search to test several values. This rate allowed the training loss to decrease smoothly and the validation accuracy to rise from 0.39 to approximately 0.70 over 10-12 epochs, without divergence or extremely slow learning.

d. stopping criteria

With the selected hyperparameters (Adam, learning rate 3×10^{-4}), the model was trained for up to 20 epochs using early stopping based on validation accuracy. Early stopping was triggered once validation accuracy stopped improving.

4. Create, explain, and provide a screenshot of your confusion matrix.



The confusion matrix is a visualization that displays the actual label of each plant class against the predicted label returned by the model. The diagonal line shows the correctly predicted classes, and the darker the color, the higher the number of predicted classes. This allows us to easily identify which categories the model performed best on and which require further improvement. The best-performing category was the Loose Silky-Bent, which the model correctly predicted 118 times. The lowest-performing category was Blackgrass, which the model correctly predicted only 11 times. Interestingly, it incorrectly labeled this category as "Loose Silky-Bent," which was the highest-performing category. Overall, the model performs well on most species, but systematically confuses visually similar grasses.

Part IV: Model Evaluation

F. Analyze the model by doing the following:

1. Evaluate the model training process and its relevant outcomes by doing the following:

a. Discuss the impact of using stopping criteria, including defining the number of epochs, and include a screenshot showing the final training epoch.

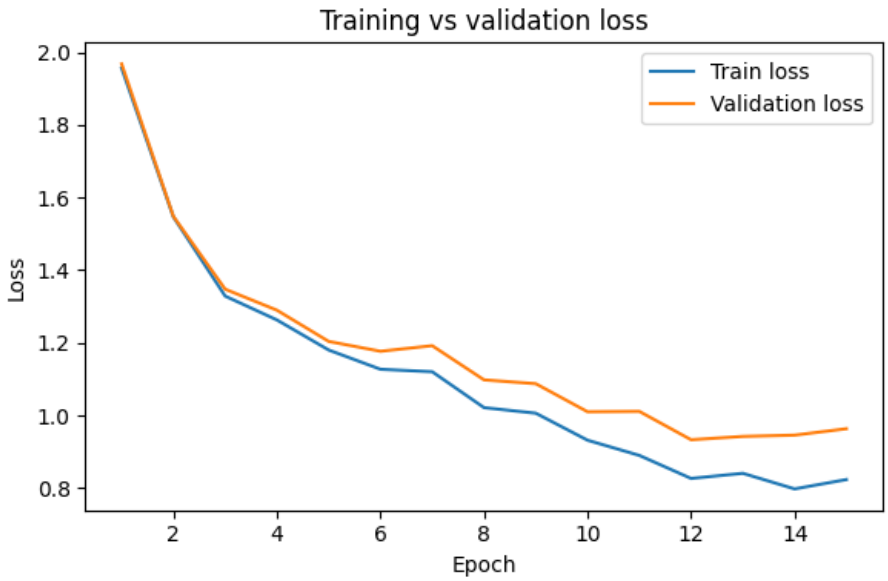
I trained the CNN for up to 20 epochs using early stopping based on validation accuracy. Training loss decreased steadily, and validation accuracy rose quickly in the first ~9 epochs, after which it plateaued. Consequently, early stopping (patience = 3) was triggered at epoch 12. This prevented wasting time on epochs that did not improve validation performance and reduced the risk of overfitting.

b. Compare the training data to the validation dataset using an evaluation metric such as loss, accuracy, F1, or mean absolute error (MAE) to assess model performance. Include an explanation.

I used accuracy as the primary evaluation metric. During training, accuracy on the validation set increased from about 0.40 in epoch 1 to around 0.71 at its peak, while training loss decreased from roughly 2.1 to 0.8. The fact that validation accuracy increased alongside decreasing training loss and did not suddenly drop while training loss continued to fall indicates the model was learning useful patterns rather than memorizing the training data.

c. Provide a visualization comparing the model's training versus validation loss. Include a screenshot.

I plotted training and validation loss per epoch on the same graph. Both curves initially decreased rapidly and then flattened out, with the validation loss closely tracking the



training loss. The gap between them stayed relatively small, which again suggests limited overfitting.

2. Assess the fitness of the model and *any* actions taken to address overfitting or underfitting.

The final model achieves approximately 70% accuracy on both the validation and test sets, which is reasonable for a 12-class plant species dataset. The similar values for training, validation, and test accuracy suggest the model is neither heavily underfitting nor heavily overfitting. To guard against overfitting, I used a relatively small CNN, normalized all images, shuffled the data, and applied early stopping when validation accuracy stopped improving.

3. Discuss the predictive accuracy of the trained network using the test set and the chosen evaluation metric from part F1b.

The trained network achieved about 70% accuracy on the held-out test set of 950 images. Since this closely matches the best validation accuracy, it indicates that the model's performance during training generalizes well to unseen data. The confusion matrix and per-class precision/recall values suggest that several species are classified with high reliability, while a few visually similar classes, such as Blackgrass and Loose Silky Bent, remain challenging.

Part V: Summary and Recommendations

G. Summarize results by doing the following:

1. Provide the code you used to save the trained network within the neural network.

```
model_path = "best_cnn_model.pth"
torch.save(net.state_dict(best_state_dict), model_path)
```

✓ 0.1s

2. Discuss the functionality of your neural network, including the impact of the network architecture.

My CNN takes a 64×64 RGB image and passes it through two convolutional–ReLU–max pooling blocks, then flattens it to 8192 features and uses one linear layer to output 12 logits (one per species). This small feature extractor, combined with a simple classifier, is expressive enough to achieve approximately 70% accuracy while keeping the number of parameters low, at roughly 103k, which helps maintain training stability and limit overfitting on the dataset.

3. **Discuss the effectiveness of the model in addressing the business problem you identified in part A1.**

The model achieved a 70% test-set accuracy, demonstrating strong performance on key species (e.g., Loose Silky Bent, Scentless Mayweed, Small-flowered Cranesbill), indicating that it can meaningfully assist in early-stage identification, albeit not perfectly. Misclassifications for species like Black Grass or Shepherd's Purse suggest that it should be used as a decision-support tool, rather than an automatic replacement for human specialists.

4. **Discuss lessons learned including how the model might be improved (i.e., if you were to deploy this model in a real-life scenario).**

I learned that CNN with basic augmentation and early stopping can already generalize reasonably well on this dataset, but that visually similar species remain challenging. In a real deployment scenario, I would recommend collecting more labeled images for weak classes and adding more substantial augmentation. I would also suggest trying a slightly deeper or pretrained backbone with regularization (dropout, weight decay) to improve model accuracy.

5. **Recommend a course of action based on your results as they relate to the research question.**

Based on the current results, the model is good enough to pilot as a supportive tool. It can be used to prioritize which plants to inspect or to pre-label images that a human then verifies. At

the same time, continued data collection and model iteration focused on the poorest-performing classes will be necessary before relying on it for fully automated weed-control decisions.

Part VI: Reporting

I. Submit a list of *all* the specific web sources you used to acquire segments of third-party code to support the application.

https://www.datacamp.com/tutorial/pytorch-cnn-tutorial

https://www.geeksforgeeks.org/deep-learning/building-a-convolutional-neural-network-using-pytorch/

REFERENCES

DataCamp. (2025). *Intermediate deep learning with PyTorch.* DataCamp. https://app.datacamp.com/learn/courses/intermediate-deep-learning-with-pytorch

GeeksforGeeks. (2025, July 23). *Standard deviation across the image channels in PyTorch*. GeeksforGeeks. **https://www.geeksforgeeks.org/computer-vision/standard-deviation-across-the-image-channels-in-pytorch/**